

## Recruitment Tasks – Technical Domain

**Task Submission Link :** <https://forms.gle/6JBrPJCsqPzWjWKy6>

**For 1st Year**

### Task 1: Signup Form with Validation and Dashboard

**Objective:** Build a small web application for user signup, validation, and dashboard view.

#### Instructions:

- Create a signup form with the following input fields:
  - **Username** → cannot be empty
  - **Email** → must be in proper format (use regex)
  - **Password** → at least 6 characters long (hash the password before storing)
- Add a **Signup button**.
- On clicking the button:
  - Validate inputs.
  - If valid → store details in **localStorage**.
- Display all user details in a **table** (columns: Username, Email, Password).

#### Brownie Subtask:

Add a **Delete button** in the dashboard table to remove user entries.

### Task 2: Personal Introduction Page

#### Objective

Build a responsive personal introduction webpage that showcases who you are while demonstrating your HTML, CSS, and JavaScript skills.

#### Instructions

Create a webpage that includes:

- Your name
- A short introduction about yourself
- Your photograph
- Your interests, hobbies, skills, or anything you'd like to share
- Your social links (optional)

Ensure the page is fully responsive for both desktop and mobile devices.

Focus on creating a clean, attractive, and interactive user interface.

#### Creative Freedom

This task is intentionally open-ended. Feel free to experiment with animations, unique layouts, custom illustrations, transitions, themes, or any creative ideas that help your personality stand out. The more creative and polished your submission, the better.

**Email**

nscc@srmist.edu.in

**Phone**

+91 8603405145  
+91 7275587001

**Address**

SRM University, Potheri SRM  
Nagar, Kattankulathur Tamil  
Nadu: 603203

### Brownie Subtask ★

Add a **Dark/Light Mode** toggle and save the selected theme using `localStorage` so it persists across page refreshes.

### Important

You are not expected to know every concept required for this task beforehand. You may research and learn new concepts while completing the task. What matters is your ability to understand the problem, learn, implement, debug, and build a working solution.

### Expected Prerequisites

You should have a basic understanding of:

- HTML
- CSS (including responsive design / media queries)
- JavaScript
- Basic DOM manipulation
- Event listeners
- `localStorage` basics

### Submission

Submit:

- The complete source code, pushed to a **proper GitHub repository**
- A **well-written README** in the repository that clearly explains the project to someone who has never seen it before, including:
  - How to run the project
  - Features implemented
  - Any additional features you added
  - Any concepts you learned while completing the task
- A short **demo video** showing the page and its interactions

*Feel free to use AI to help you build this, but make sure you have a proper understanding of the code it writes — you should be able to explain any function used and what it does.*

### Email

nscc@srmist.edu.in

### Phone

+91 8603405145  
+91 7275587001

### Address

SRM University, Potheri SRM  
Nagar, Kattankulathur Tamil  
Nadu: 603203

## For 2nd Year

### General Requirements

The objective of these tasks is to evaluate your understanding of full-stack web development, including frontend development, backend development, database management, API design, authentication, validation, and application logic.

For all tasks:

- You are expected to build a **functional full-stack application**.
- You may use any suitable frontend framework/library such as React, Vue, or vanilla JavaScript.
- You should use suitable backend framework such as Node.js + Express.
- You may use databases such as MongoDB, PostgreSQL, MySQL, or an equivalent database.
- Your application should follow a clean and maintainable project structure.
- Proper validation and error handling are expected.

### Backend/Database Restriction

**The use of Supabase and Firebase is strictly prohibited for these tasks.**

The purpose of this restriction is to evaluate your understanding of implementing the backend, database operations, APIs, authentication, and server-side business logic yourself. You are expected to implement your own backend/API layer using an appropriate backend framework.

## Task 1: Library Book Issue & Return Management System

### Objective

Develop a full-stack web application to streamline the management of library books by enabling librarians to issue and return books using QR code scanning. The system should maintain accurate records of book availability, issue/return history, and provide real-time tracking of borrowed books.

### Instructions

#### 1. Frontend — Librarian Side

Create an interface that allows librarians to:

- Add and manage book records with: Book Title, Author, ISBN/Book ID, Category, Total Copies
- Generate a unique QR code for each book
- Scan a book's QR code using the device camera
- Issue and return books using QR scanning
- Display the current status of books: Available / Issued
- Search and filter books by Title, Author, Category, Availability

## 2. Backend — Server Side

Implement a backend that:

- Stores book information in a database
- Stores issue/return transaction records
- Verifies the scanned book before processing a transaction
- Updates book availability when a book is issued or returned
- Records issue timestamp, return timestamp, and borrower information
- Prevents issuing a book that is already checked out
- Maintains a complete history of all issue/return transactions
- Implements proper validation and error handling for invalid book IDs, invalid QR codes, duplicate operations, invalid return operations, and unavailable books

## 3. Data Export

Provide functionality to export the complete issue/return history as CSV or Excel, containing: Book Title, Author, Book ID, Issued To (User ID/Name), Issue Timestamp, Return Timestamp, Current Status.

### Brownie Subtask ★

Develop an **Admin Dashboard** that provides:

- Total Books, Available Books, Issued Books, Overdue Books
- List of currently issued books with borrower details
- Search and filter for transactions
- Downloadable issue/return reports
- Bonus: calculate and display the number of days a book is overdue

### Optional Bonus ★ — AI 🤖

Add an AI-powered feature that provides meaningful value to the application, such as:

- AI-powered smart search
- Library assistant/chatbot
- Automatic book categorization
- Book recommendations
- Natural-language search
- Any other useful AI feature of your choice

This is completely optional and will not be required for the core task, but the AI implementation should provide actual functionality rather than simply being added for demonstration.

### Important

You are not expected to know every concept required for this task beforehand. You may research and learn new concepts while completing the task. What matters is your ability to understand the problem, learn, implement, debug, and build a working solution.

## Expected Prerequisites

You should have a basic understanding of:

- HTML, CSS, JavaScript
- A frontend framework/library (React, Vue, or vanilla JS)
- A backend framework (Node.js + Express)
- A database (MongoDB, PostgreSQL, MySQL, or equivalent)
- REST API design
- QR code generation/scanning basics
- Validation and error handling

## Submission

Submit:

- The complete source code, pushed to a **proper GitHub repository**
- A working, deployed version of the application (frontend and backend)
- A **well-written README** in the repository that clearly explains the project to someone who has never seen it before, including:
  - How to run the project (setup instructions, database configuration)
  - APIs implemented
  - Features implemented
  - Any additional features you added
  - Important implementation decisions
  - Any concepts you learned while completing the task
- A short **demo video** showing the application in action

*Feel free to use AI to help you build this, but make sure you have a proper understanding of the code it writes — you should be able to explain any function used and what it does.*

## Task 2: QR-Based Geo-Tagged Attendance Management System

### Objective

Develop a full-stack web application that enables organizers to manage events and securely record attendance using QR code scanning combined with geolocation verification.

### Instructions

#### 1. Frontend — User Side

Implement:

- User authentication using Google OAuth 2.0 or JWT-based authentication
- Display of upcoming events with details (name, venue, date, time)
- QR code scanning using the device camera
- Browser/device geolocation access
- Location verification before marking attendance
- Attendance confirmation after successful verification

#### 2. Frontend — Organizer Side

Organizers should be able to:

- Create, edit, and delete events, defining: event name, venue, date, time, geofence radius
- Generate a unique QR code for each event
- View attendees for each event
- Search and filter attendees

#### 3. Backend — Server Side

Implement a backend that:

- Stores event information, users/participants, and attendance records
- Validates QR codes
- Verifies the user's reported location falls within the configured geofence radius of the event venue before accepting attendance (advanced anti-GPS-spoofing mechanisms are not required)
- Prevents duplicate attendance submissions
- Records attendance timestamps
- Maintains organizer-wise event records and ensures only authorized organizers can manage their own events
- Implements proper validation and error handling

## 4. Data Export

Allow organizers to export attendance records as CSV or Excel, containing: Name, Registration ID, Email, Attendance Status, Timestamp.

### Brownie Subtask ★

Implement **real-time attendance tracking** for organizers. The dashboard should display:

- Total registrations, total attendees, attendance percentage
- Live/recent attendance updates
- Downloadable attendance reports

### Optional Bonus ★ — AI 🤖

Add an AI-powered feature that provides meaningful value to the application, such as:

- AI event assistant
- Natural-language event search
- Attendance insights
- Automated event description generation
- Smart attendee analytics
- Personalized event recommendations
- Any other useful AI feature of your choice

This is completely optional and will not be required for the core task.

### Important

You are not expected to know every concept required for this task beforehand. You may research and learn new concepts while completing the task. What matters is your ability to understand the problem, learn, implement, debug, and build a working solution.

### Expected Prerequisites

You should have a basic understanding of:

- HTML, CSS, JavaScript
- A frontend framework/library (React, Vue, or vanilla JS)
- A backend framework (Node.js + Express)
- A database (MongoDB, PostgreSQL, MySQL, or equivalent)
- REST API design
- Authentication (Google OAuth 2.0 or JWT)
- Browser geolocation APIs
- QR code generation/scanning basics

## Submission

Submit:

- The complete source code, pushed to a **proper GitHub repository**
- A working, deployed version of the application (frontend and backend)
- A **well-written README** in the repository that clearly explains the project to someone who has never seen it before, including:
  - How to run the project (setup instructions, database configuration)
  - APIs implemented
  - Features implemented
  - Any additional features you added
  - Important implementation decisions
  - Any concepts you learned while completing the task
- A short **demo video** showing the application in action

*Feel free to use AI to help you build this, but make sure you have a proper understanding of the code it writes — you should be able to explain any function used and what it does.*